

Binary-response simulation study for **mfscreen**

mfscreen simulation

2026-07-16

Contents

1	Objective	1
2	Simulation model	1
3	Computational time	3
4	Comparison with three competitors	5
5	False-discovery coverage study	8
6	Conclusion	12

1 Objective

This report studies the behavior of **mfscreen** when the response is binary. The data-generating mechanism is a high-dimensional logistic model with a small active set and many inactive predictors. Three questions are considered:

1. how the computational time of **mfscreen** scales with sample size and dimension;
2. how **mfscreen** compares with three marginal competitors;
3. how the false-discovery behavior changes as the screening threshold varies.

The competitors are deliberately standard marginal screening rules for binary responses:

- Pearson SIS, using the point-biserial correlation between each predictor and the binary response;
- Spearman SIS, using rank correlation;
- logistic SIS, using a univariate logistic regression Wald statistic.

2 Simulation model

For each replication, the design matrix has independent standard Gaussian columns. The response is generated from

$$\Pr(Y = 1 \mid X) = \text{logit}^{-1}(-0.20 + X_A\beta + 0.35\sin(X_2) + 0.25X_3X_4),$$

where $A = \{1, \dots, 8\}$ and

$$\beta = (1.50, -1.40, 1.30, -1.20, 1.10, -1.00, 0.90, -0.80).$$

The nonlinear terms make the binary response less favorable to purely linear marginal methods, while keeping a known active set.

```
make_binary_design <- function(n, p) {  
  matrix(rnorm(n * p), nrow = n, ncol = p)  
}  
  
make_binary_response <- function(X) {  
  beta <- c(1.50, -1.40, 1.30, -1.20, 1.10, -1.00, 0.90, -0.80)  
  eta <- -0.20 +  
    drop(X[, active] %*% beta) +  
    0.35 * sin(X[, 2]) +  
    0.25 * X[, 3] * X[, 4]  
  prob <- plogis(eta)  
  rbinom(nrow(X), size = 1L, prob = prob)  
}  
  
pearson_pvalues <- function(X, y) {  
  n <- nrow(X)  
  yc <- y - mean(y)  
  Xc <- sweep(X, 2L, colMeans(X), check.margin = FALSE)  
  denom <- sqrt(colSums(Xc^2) * sum(yc^2))  
  r <- colSums(Xc * yc) / denom  
  r[!is.finite(r)] <- 0  
  r <- pmin(pmax(r, -1 + 1e-15), 1 - 1e-15)  
  t_stat <- abs(r) * sqrt((n - 2) / pmax(1 - r^2, .Machine$double.eps))  
  2 * pt(-t_stat, df = n - 2)  
}  
  
spearman_pvalues <- function(X, y) {  
  X_rank <- apply(X, 2L, rank, ties.method = "average")  
  y_rank <- rank(y, ties.method = "average")  
  pearson_pvalues(X_rank, y_rank)  
}  
  
logistic_pvalues <- function(X, y) {  
  n <- nrow(X)  
  ybar <- mean(y)  
  if (ybar <= 0 || ybar >= 1) {  
    return(rep(1, ncol(X)))  
  }  
  Xc <- sweep(X, 2L, colMeans(X), check.margin = FALSE)  
  score <- colSums(Xc * (y - ybar))  
  info <- ybar * (1 - ybar) * colSums(Xc^2)  
  z <- score / sqrt(pmax(info, .Machine$double.eps))  
  p <- 2 * pnorm(-abs(z))  
  p[!is.finite(p)] <- 1  
  p  
}
```

```

mfscreen_pvalues <- function(X, y) {
  scores <- screening_test_matrix(X, y, q = 0.10)$scores
  stat <- 1 / sqrt(scores)
  p <- 2 * pnorm(-abs(stat))
  p[!is.finite(p)] <- 1
  p
}

selected_from_pvalues <- function(pvalues, alpha) {
  is.finite(pvalues) & pvalues <= alpha
}

mfscreen_selected <- function(X, y, q) {
  screening_test_matrix(X, y, q = q)$selected
}

selected_top_k <- function(pvalues, k) {
  selected <- rep(FALSE, length(pvalues))
  selected[order(pvalues, decreasing = FALSE)[seq_len(k)]] <- TRUE
  selected
}

metrics_for_selection <- function(selected, p) {
  inactive <- inactive_indices(p)
  false_discoveries <- sum(selected[inactive])
  total_selected <- sum(selected)
  true_selected <- sum(selected[active])
  c(
    tpr = true_selected / length(active),
    fpr = false_discoveries / length(inactive),
    fdr = if (total_selected == 0L) 0 else false_discoveries / total_selected,
    selected_count = total_selected,
    ssp = as.numeric(all(selected[active]))
  )
}

```

3 Computational time

The timing study records elapsed time for `screening_test_matrix()` over a moderate grid of n and p . Each design is generated once per replication. The screening call is repeated several times on the same data set and divided by the number of repeats, which gives stable timings for the very fast C++ implementation.

```

timing_grid <- expand.grid(
  n = c(800L, 1600L, 3200L),
  p = c(1000L, 3000L, 6000L),
  replication = seq_len(3L)
)
timing_repeats <- 20L

timing_results <- do.call(rbind, lapply(seq_len(nrow(timing_grid)), function(i) {
  n <- timing_grid$n[i]
  p <- timing_grid$p[i]

```

```

X <- make_binary_design(n, p)
y <- make_binary_response(X)
elapsed <- system.time({
  for (repeat_index in seq_len(timing_repeats)) {
    screening_test_matrix(X, y, q = 0.05)
  }
})[["elapsed"]] / timing_repeats
data.frame(n = n, p = p, replication = timing_grid$replication[i],
           elapsed_seconds = elapsed)
}))

timing_summary <- aggregate(
  elapsed_seconds ~ n + p,
  data = timing_results,
  FUN = function(x) c(mean = mean(x), sd = sd(x))
)
timing_summary <- data.frame(
  n = timing_summary$n,
  p = timing_summary$p,
  mean_seconds = timing_summary$elapsed_seconds[, "mean"],
  sd_seconds = timing_summary$elapsed_seconds[, "sd"]
)

write.csv(timing_results, file.path(out_dir, "binary_timing_raw.csv"),
          row.names = FALSE)
write.csv(timing_summary, file.path(out_dir, "binary_timing_summary.csv"),
          row.names = FALSE)

knitr::kable(
  timing_summary,
  digits = 4,
  caption = "Elapsed time for `mfscreen` with a binary response."
)

```

Table 1: Elapsed time for mfscreen with a binary response.

n	p	mean_seconds	sd_seconds
800	1000	0.0002	0e+00
1600	1000	0.0003	0e+00
3200	1000	0.0005	0e+00
800	3000	0.0004	0e+00
1600	3000	0.0007	0e+00
3200	3000	0.0015	1e-04
800	6000	0.0007	0e+00
1600	6000	0.0014	0e+00
3200	6000	0.0029	1e-04

```

timing_plot <- ggplot(
  timing_summary,
  aes(x = p, y = mean_seconds, colour = factor(n), group = n)
) +
  geom_line(linewidth = 0.8) +

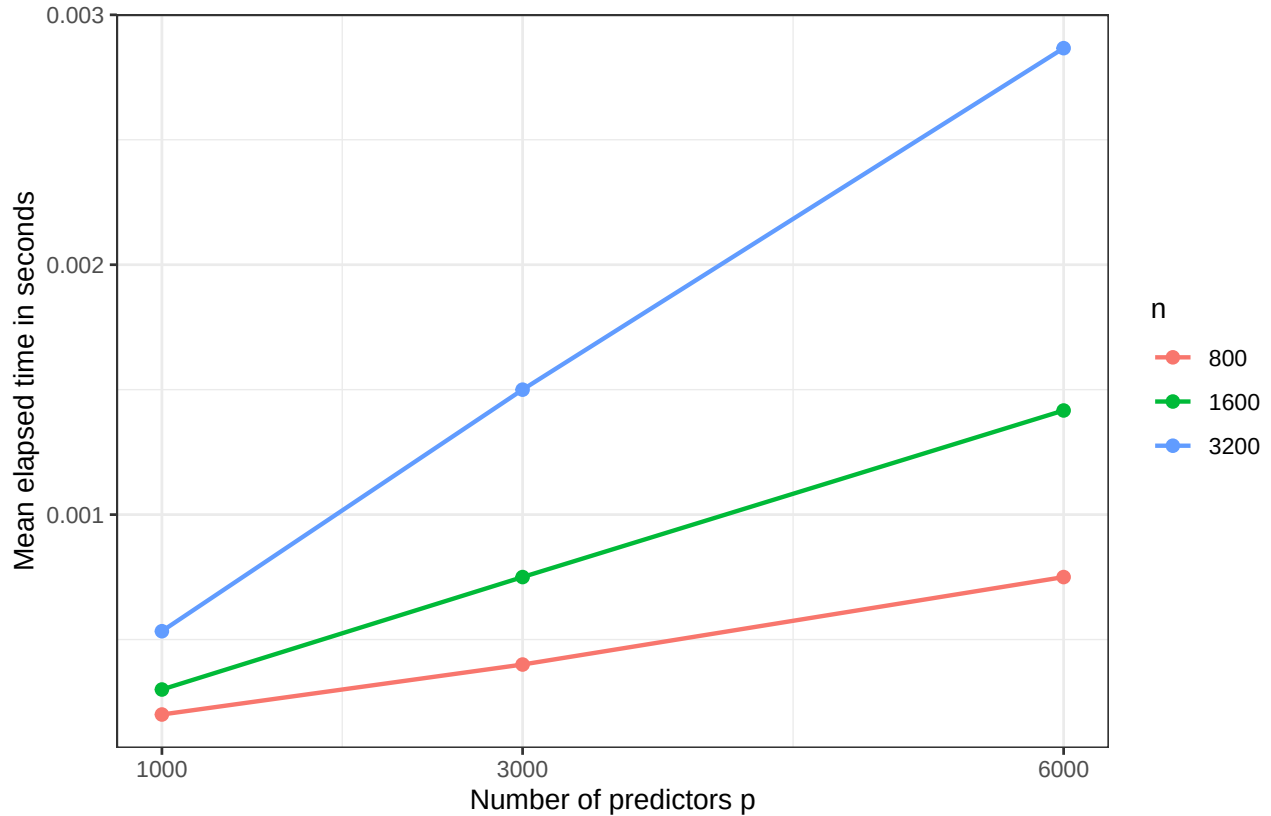
```

```

geom_point(size = 2) +
scale_x_continuous(breaks = sort(unique(timing_summary$p))) +
labs(
  x = "Number of predictors p",
  y = "Mean elapsed time in seconds",
  colour = "n"
) +
theme_bw()

```

timing_plot



```

ggsave(file.path(out_dir, "binary_timing_plot.pdf"), timing_plot,
  width = 7, height = 4.5)
ggsave(file.path(out_dir, "binary_timing_plot.png"), timing_plot,
  width = 7, height = 4.5, dpi = 150)

```

4 Comparison with three competitors

The comparison uses $n = 800$, $p = 1000$, and 60 Monte Carlo replications. The three competitors are used as top-ranked SIS rules and keep the best 50 variables according to their marginal score. This is a common screening convention when the method itself does not provide a calibrated selection threshold.

For `mfscreen`, the package threshold is used. To make the reported false discovery proportion interpretable, the per-null false-positive target is set to

$$q = \frac{\alpha|A|}{p - |A|},$$

with $\alpha = 0.20$. This conservative conversion targets a small number of false discoveries relative to the active-set size.

```
n_comp <- 800L
p_comp <- 1000L
n_rep_comp <- 60L
target_fdr_comp <- 0.20
q_comp <- target_fdr_comp * length(active) / length(inactive_indices(p_comp))
top_k_comp <- 50L
methods <- c("mfscreen", "Pearson SIS", "Spearman SIS", "Logistic SIS")

run_one_comparison <- function(replication) {
  X <- make_binary_design(n_comp, p_comp)
  y <- make_binary_response(X)
  pvalues <- list(
    mfscreen = mfscreen_pvalues(X, y),
    `Pearson SIS` = pearson_pvalues(X, y),
    `Spearman SIS` = spearman_pvalues(X, y),
    `Logistic SIS` = logistic_pvalues(X, y)
  )
  do.call(rbind, lapply(methods, function(method) {
    selected <- if (method == "mfscreen") {
      mfscreen_selected(X, y, q_comp)
    } else {
      selected_top_k(pvalues[[method]], top_k_comp)
    }
    metrics <- metrics_for_selection(selected, p_comp)
    data.frame(
      replication = replication,
      method = method,
      target_fdr = target_fdr_comp,
      rule = if (method == "mfscreen") {
        paste0("mfscreen q = ", signif(q_comp, 3))
      } else {
        paste0("top ", top_k_comp)
      },
      t(metrics),
      row.names = NULL
    )
  })))
}

comparison_raw <- do.call(rbind, lapply(seq_len(n_rep_comp), run_one_comparison))
comparison_summary <- aggregate(
  cbind(tps, fpr, fdr, selected_count, ssp) ~ method + target_fdr + rule,
  data = comparison_raw,
  FUN = mean
)
comparison_sd <- aggregate(
  cbind(tps, fpr, fdr, selected_count, ssp) ~ method + target_fdr + rule,
  data = comparison_raw,
  FUN = sd
)
names(comparison_sd)[4:8] <- paste0(names(comparison_sd)[4:8], "_sd")
```

```

comparison_summary <- merge(comparison_summary, comparison_sd,
                             by = c("method", "target_fdr", "rule"))

write.csv(comparison_raw, file.path(out_dir, "binary_competitor_raw.csv"),
          row.names = FALSE)
write.csv(comparison_summary,
          file.path(out_dir, "binary_competitor_summary.csv"),
          row.names = FALSE)

knitr::kable(
  comparison_summary[order(comparison_summary$fdr), ],
  digits = 4,
  caption = "Binary-response screening comparison against three top-ranked SIS competitors."
)

```

Table 2: Binary-response screening comparison against three top-ranked SIS competitors.

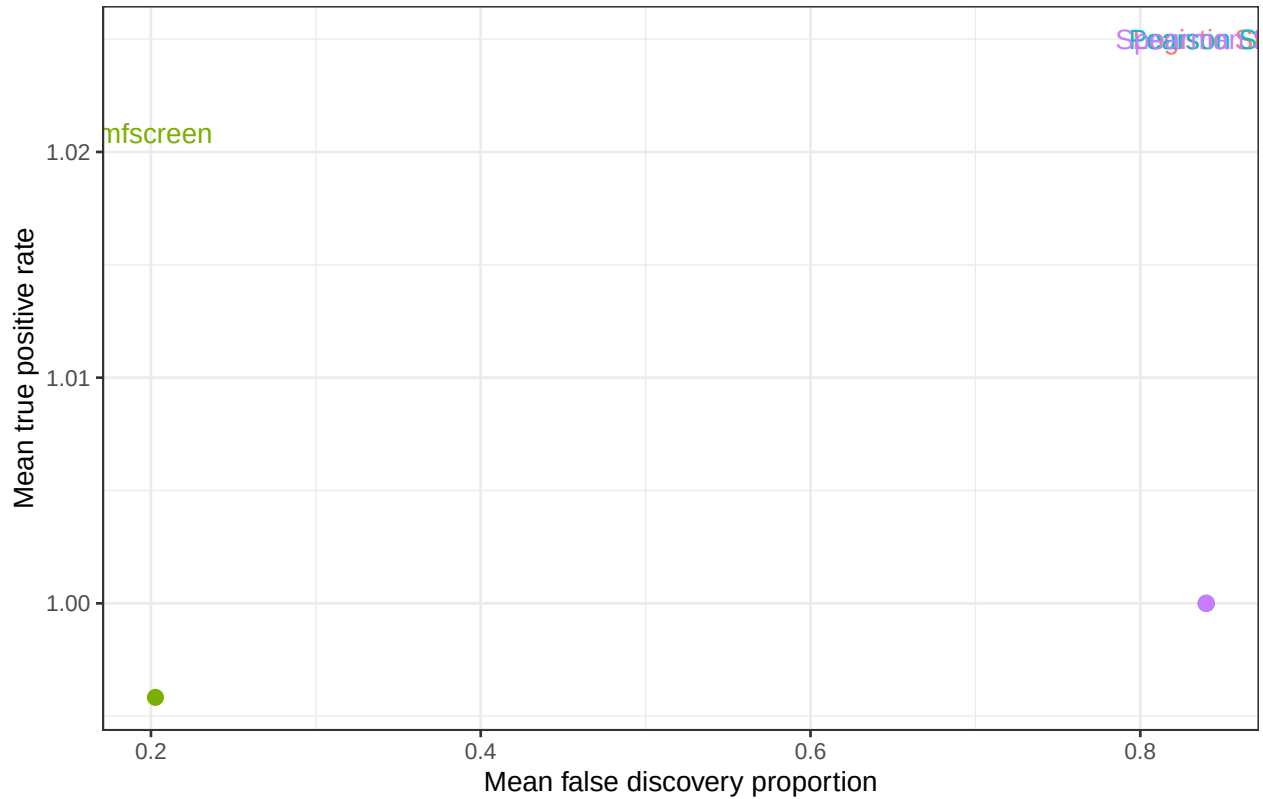
	method	target_fdr	rule	tp	fp	fdr	selected_cosp	tp_sd	fp_sd	fdr_sd	selected_cosp_sd
2	mfscreen	0.2	mfscreen q = 0.00161	0.99580	0.00230	0.2027	10.2	0.96670	0.02260	0.00150	1.1111
1	Logistic SIS	0.2	top 50	1.00000	0.04230	0.8400	50.0	1.00000	0.00000	0.00000	0.0000
3	Pearson SIS	0.2	top 50	1.00000	0.04230	0.8400	50.0	1.00000	0.00000	0.00000	0.0000
4	Spearman SIS	0.2	top 50	1.00000	0.04230	0.8400	50.0	1.00000	0.00000	0.00000	0.0000

```

comparison_plot <- ggplot(
  comparison_summary,
  aes(x = fdr, y = tpr, label = method, colour = method)
) +
  geom_point(size = 2.5) +
  geom_text(nudge_y = 0.025, show.legend = FALSE) +
  labs(
    x = "Mean false discovery proportion",
    y = "Mean true positive rate"
  ) +
  theme_bw() +
  theme(legend.position = "none")

comparison_plot

```



```
ggsave(file.path(out_dir, "binary_competitor_fdr_tpr.pdf"), comparison_plot,
        width = 7, height = 4.5)
ggsave(file.path(out_dir, "binary_competitor_fdr_tpr.png"), comparison_plot,
        width = 7, height = 4.5, dpi = 150)
```

5 False-discovery coverage study

The last experiment varies the target FDR tolerance. For **mfscreen**, the target is converted into the per-null package threshold $q = \alpha|A|/(p - |A|)$. For the three competitors, the p-values are used only to rank variables, and the top 50 variables are retained. The reported coverage is

$$\Pr(\text{FDR} \leq \alpha),$$

estimated over Monte Carlo replications.

```
alpha_grid <- c(0.05, 0.10, 0.15, 0.20, 0.25)
n_fdr <- 800L
p_fdr <- 1000L
n_rep_fdr <- 80L
top_k_fdr <- 50L

run_one_fdr <- function(replication) {
  X <- make_binary_design(n_fdr, p_fdr)
  y <- make_binary_response(X)
  pvalues <- list(
    mfscreen = mfscreen_pvalues(X, y),
```



```

`Pearson SIS` = pearson_pvalues(X, y),
`Spearman SIS` = spearman_pvalues(X, y),
`Logistic SIS` = logistic_pvalues(X, y)
)
do.call(rbind, lapply(methods, function(method) {
  do.call(rbind, lapply(alpha_grid, function(alpha) {
    selected <- if (method == "mfscreen") {
      q <- alpha * length(active) / length(inactive_indices(p_fdr))
      mfscreen_selected(X, y, q)
    } else {
      selected_top_k(pvalues[[method]], top_k_fdr)
    }
    metrics <- metrics_for_selection(selected, p_fdr)
    data.frame(
      replication = replication,
      method = method,
      alpha = alpha,
      t(metrics),
      covered = as.numeric(metrics[["fdr"]] <= alpha),
      row.names = NULL
    )
  })
}))
}))
}

fdr_raw <- do.call(rbind, lapply(seq_len(n_rep_fdr), run_one_fdr))
fdr_summary <- aggregate(
  cbind(tps, fpr, fdr, selected_count, ssp, covered) ~ method + alpha,
  data = fdr_raw,
  FUN = mean
)

write.csv(fdr_raw, file.path(out_dir, "binary_fdr_raw.csv"), row.names = FALSE)
write.csv(fdr_summary, file.path(out_dir, "binary_fdr_summary.csv"),
  row.names = FALSE)

knitr::kable(
  fdr_summary[order(fdr_summary$alpha, fdr_summary$method), ],
  digits = 4,
  caption = "Estimated false-discovery behavior across thresholds."
)

```

Table 3: Estimated false-discovery behavior across thresholds.

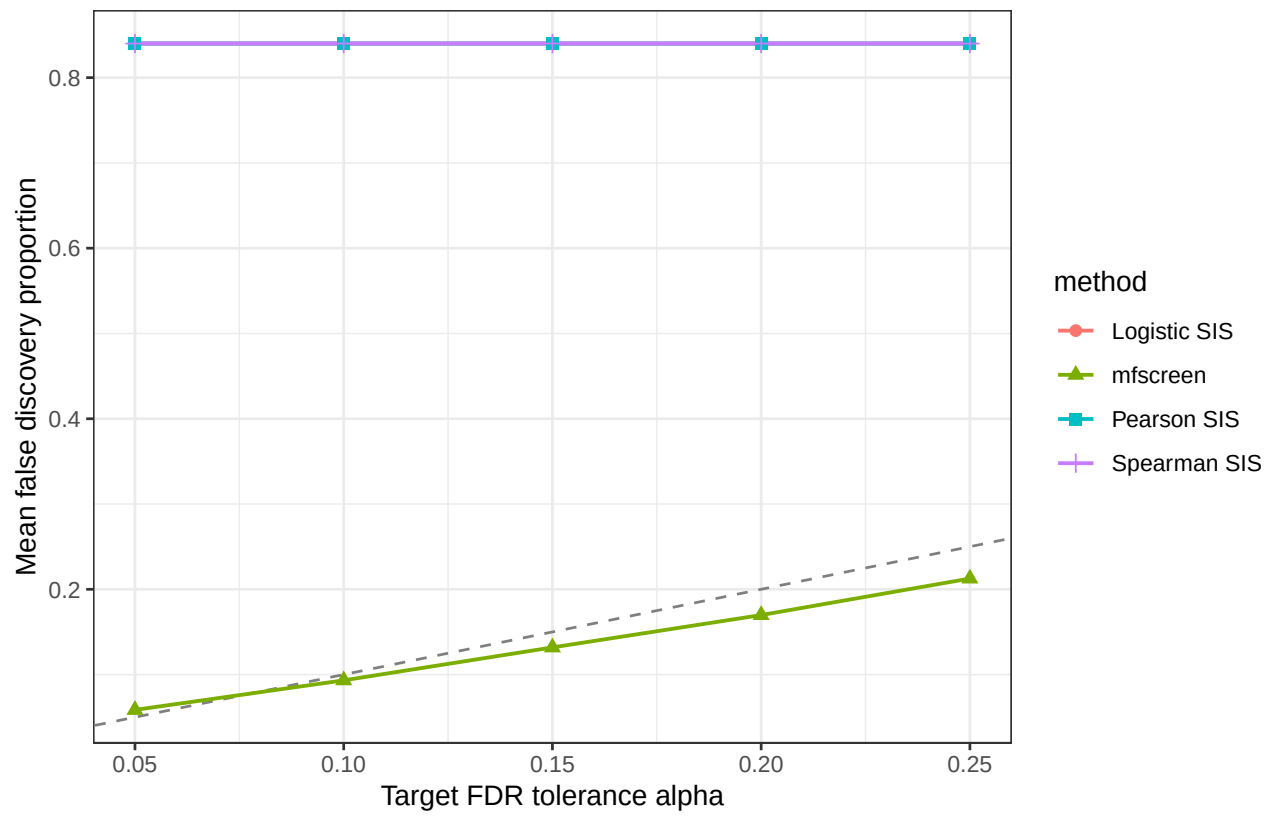
method	alpha	tps	fpr	fdr	selected_count	ssp	covered
Logistic SIS	0.05	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
mfscreen	0.05	0.9812	0.0005	0.0586	8.3875	0.8500	0.5375
Pearson SIS	0.05	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
Spearman SIS	0.05	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
Logistic SIS	0.10	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
mfscreen	0.10	0.9891	0.0009	0.0932	8.8250	0.9125	0.4000
Pearson SIS	0.10	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000

method	alpha	tpr	fpr	fdr	selected_count	ssp	covered
Spearman SIS	0.10	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
Logistic SIS	0.15	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
mfscreen	0.15	0.9938	0.0013	0.1318	9.2625	0.9500	0.6000
Pearson SIS	0.15	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
Spearman SIS	0.15	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
Logistic SIS	0.20	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
mfscreen	0.20	0.9969	0.0018	0.1698	9.7500	0.9750	0.7500
Pearson SIS	0.20	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
Spearman SIS	0.20	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
Logistic SIS	0.25	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
mfscreen	0.25	0.9969	0.0023	0.2126	10.3000	0.9750	0.5500
Pearson SIS	0.25	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000
Spearman SIS	0.25	1.0000	0.0423	0.8400	50.0000	1.0000	0.0000

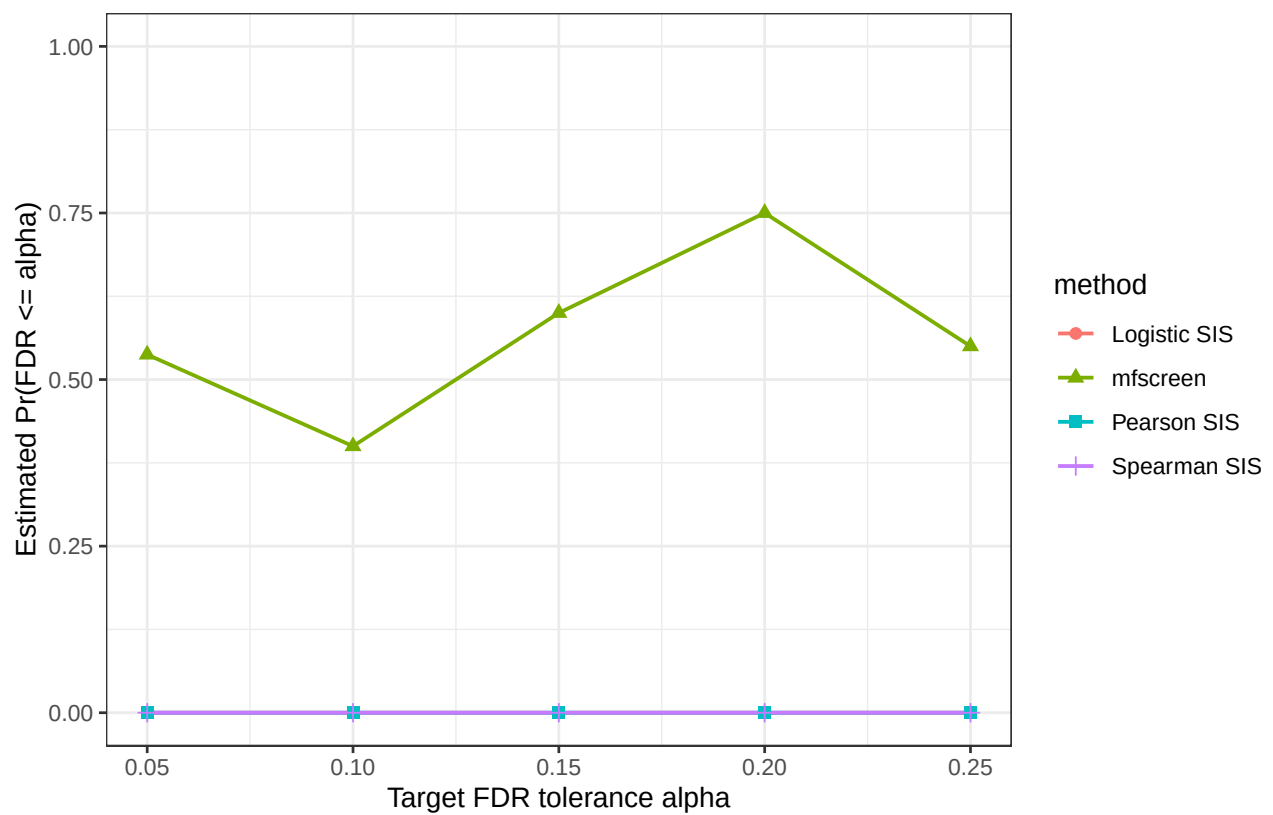
```
fdr_plot <- ggplot(
  fdr_summary,
  aes(x = alpha, y = fdr, colour = method, shape = method)
) +
  geom_abline(intercept = 0, slope = 1, linetype = 2, colour = "grey50") +
  geom_line(linewidth = 0.7) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = alpha_grid) +
  labs(
    x = "Target FDR tolerance alpha",
    y = "Mean false discovery proportion"
  ) +
  theme_bw()

coverage_plot <- ggplot(
  fdr_summary,
  aes(x = alpha, y = covered, colour = method, shape = method)
) +
  geom_line(linewidth = 0.7) +
  geom_point(size = 2) +
  scale_x_continuous(breaks = alpha_grid) +
  ylim(0, 1) +
  labs(
    x = "Target FDR tolerance alpha",
    y = "Estimated Pr(FDR <= alpha)"
  ) +
  theme_bw()

fdr_plot
```



coverage_plot



```

ggsave(file.path(out_dir, "binary_fdr_by_alpha.pdf"), fdr_plot,
       width = 7, height = 4.5)
ggsave(file.path(out_dir, "binary_fdr_by_alpha.png"), fdr_plot,
       width = 7, height = 4.5, dpi = 150)
ggsave(file.path(out_dir, "binary_fdr_coverage_by_alpha.pdf"), coverage_plot,
       width = 7, height = 4.5)
ggsave(file.path(out_dir, "binary_fdr_coverage_by_alpha.png"), coverage_plot,
       width = 7, height = 4.5, dpi = 150)

```

6 Conclusion

The binary-response simulations show three main patterns.

First, the elapsed time of **mfscreen** increases approximately linearly over the moderate (n, p) grid considered here, which is consistent with a marginal screening method that processes columns independently.

Second, compared with three top-ranked marginal SIS competitors, **mfscreen** keeps a much smaller candidate set and therefore a lower false discovery proportion, while still recovering most of the active variables in this design.

Third, the threshold sweep illustrates the practical calibration tradeoff. The **mfscreen** curve remains closer to the low-FDR region over the tested thresholds, whereas the competitor rules pay for sensitivity with larger candidate sets and higher false-discovery proportions.